

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №5

Студент Трофимов А. В.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка _____

Задание №1

В классе `FunctionPoint` были переопределены следующие методы:

`String toString()`: который возвращает текстовое описание точки.

```
41     @Override
42     public String toString() {
43         return "(" + x + "; " + y + ")";
44     }
```

`boolean equals(Object o)`: который возвращает `true` тогда и только тогда, когда переданный объект также является точкой и его координаты в точности совпадают с координатами объекта, у которого вызывается метод.

```
46     @Override
47     public boolean equals(Object o) {
48         if (this == o) return true;
49         if (!(o instanceof FunctionPoint)) return false;
50
51         FunctionPoint that = (FunctionPoint) o;
52         return Math.abs(this.x - that.x) < 1e-9 &&
53             Math.abs(this.y - that.y) < 1e-9;
54     }
```

`int hashCode()`: который возвращает значение хэш-кода для объекта точки.

```
56     @Override
57     public int hashCode() {
58         return Objects.hash(x, y);
59     }
```

`Object clone()`: который возвращает объект-копию для объекта точки.

```
61     @Override
62     public FunctionPoint clone(){
63         return new FunctionPoint(this.x, this.y);
64     }
65 }
```

Задание №2

В классе `ArrayTabulatedFunction` были переопределены следующие методы:

`String toString()`: который возвращает описание табулированной функции.

```
197     @Override
198     public String toString() {
199         StringBuilder sb = new StringBuilder();
200         sb.append(str: "{");
201         for (int i = 0; i < pointsCount; i++) {
202             if (i > 0) sb.append(str: ", ");
203             sb.append(points[i].toString());
204         }
205         sb.append(str: "}");
206         return sb.toString();
207     }
```

`boolean equals(Object o)`: который возвращает `true` тогда и только тогда, когда переданный объект также является табулированной функцией (реализует интерфейс `TabulatedFunction`) и её набор точек в точности совпадает с набором точек функции, у которой вызывается метод. В случае если переданный объект является экземпляром класса `ArrayTabulatedFunction`, метод напрямую обращается к элементам переданного объекта.

```
209     @Override
210     public boolean equals(Object o) {
211         if (this == o) return true;
212
213         if (o instanceof TabulatedFunction) {
214             TabulatedFunction other = (TabulatedFunction) o;
215
216             if (this.getPointsCount() != other.getPointsCount()) {
217                 return false;
218             }
219
220             if (o instanceof ArrayTabulatedFunction) {
221                 ArrayTabulatedFunction arrayOther = (ArrayTabulatedFunction) o;
222
223                 for (int i = 0; i < pointsCount; i++) {
224                     if (!this.points[i].equals(arrayOther.points[i])) {
225                         return false;
226                     }
227                 }
228                 return true;
229             } else {
230                 for (int i = 0; i < pointsCount; i++) {
231                     FunctionPoint thisPoint = this.getPoint(i);
232                     FunctionPoint otherPoint = other.getPoint(i);
233
234                     if (!thisPoint.equals(otherPoint)) {
235                         return false;
236                     }
237                 }
238                 return true;
239             }
240         }
241         return false;
242     }
```

int hashCode(): который возвращает значение хэш-кода для объекта табулированной функции.

```
244     @Override
245     public int hashCode() {
246         int hash = pointsCount;
247         for (int i = 0; i < pointsCount; i++) {
248             hash ^= points[i].hashCode();
249         }
250         return hash;
251     }
```

Object clone(): который возвращает объект-копию для объекта табулированной функции.

```
253     @Override
254     public Object clone() {
255         try {
256             ArrayTabulatedFunction clone = (ArrayTabulatedFunction) super.clone();
257
258             clone.points = new FunctionPoint[this.points.length];
259             for (int i = 0; i < this.pointsCount; i++) {
260                 clone.points[i] = (FunctionPoint) this.points[i].clone();
261             }
262
263             clone.pointsCount = this.pointsCount;
264
265             return clone;
266         } catch (CloneNotSupportedException e) {
267             throw new AssertionError();
268         }
269     }
270 }
```

Задание №3

В классе `LinkedListTabulatedFunction` были переопределены следующие методы:

`String toString()`: который возвращает описание табулированной функции.

```
212 @Override
213 public String toString() {
214     StringBuilder sb = new StringBuilder();
215     sb.append(str: "{");
216     Node current = head.next;
217     int count = 0;
218
219     while (current != head) {
220         if (count > 0) sb.append(str: ", ");
221         sb.append(current.point.toString());
222         current = current.next;
223         count++;
224     }
225     sb.append(str: "}");
226     return sb.toString();
227 }
```

`boolean equals(Object o)`: который возвращает `true` тогда и только тогда, когда переданный объект также является табулированной функцией (реализует интерфейс `TabulatedFunction`) и её набор точек в точности совпадает с набором точек функции, у которой вызывается метод. В случае если переданный объект является экземпляром класса `LinkedListTabulatedFunction`, метод напрямую обращается к элементам переданного объекта.

```
229 @Override
230 public boolean equals(Object o) {
231     if (this == o) return true;
232
233     if (o instanceof TabulatedFunction) {
234         TabulatedFunction other = (TabulatedFunction) o;
235
236         if (this.getPointsCount() != other.getPointsCount()) {
237             return false;
238         }
239
240         if (o instanceof LinkedListTabulatedFunction) {
241             LinkedListTabulatedFunction listOther = (LinkedListTabulatedFunction) o;
242
243             Node thisCurrent = this.head.next;
244             Node otherCurrent = listOther.head.next;
245
246             while (thisCurrent != this.head && otherCurrent != listOther.head) {
247                 if (!thisCurrent.point.equals(otherCurrent.point)) {
248                     return false;
249                 }
250                 thisCurrent = thisCurrent.next;
251                 otherCurrent = otherCurrent.next;
252             }
253             return true;
254         } else {
255             for (int i = 0; i < size; i++) {
256                 FunctionPoint thisPoint = this.getPoint(i);
257                 FunctionPoint otherPoint = other.getPoint(i);
258
259                 if (!thisPoint.equals(otherPoint)) {
260                     return false;
261                 }
262             }
263             return true;
264         }
265     }
266     return false;
267 }
```

int hashCode(): который возвращает значение хэш-кода для объекта табулированной функции.

```
269     @Override
270     public int hashCode() {
271         int hash = size;
272         Node current = head.next;
273
274         while (current != head) {
275             hash ^= current.point.hashCode();
276             current = current.next;
277         }
278         return hash;
279     }
```

Object clone(): который возвращает объект-копию для объекта табулированной функции.

```
281     @Override
282     public Object clone() {
283         LinkedListTabulatedFunction clone = new LinkedListTabulatedFunction();
284
285         if (size > 0) {
286             Node current = this.head.next;
287             FunctionPoint[] pointsArray = new FunctionPoint[size];
288
289             for (int i = 0; i < size; i++) {
290                 pointsArray[i] = (FunctionPoint) current.point.clone();
291                 current = current.next;
292             }
293
294             Node prevNode = clone.head;
295             for (int i = 0; i < size; i++) {
296                 Node newNode = new Node(pointsArray[i]);
297
298                 newNode.prev = prevNode;
299                 prevNode.next = newNode;
300
301                 prevNode = newNode;
302             }
303
304             prevNode.next = clone.head;
305             clone.head.prev = prevNode;
306             clone.size = size;
307         }
308
309         return clone;
310     }
311 }
```

Задание №4

Сделано так, чтобы все объекты типа TabulatedFunction были клонируемыми с точки зрения JVM и внесен метод clone() в этот интерфейс.

```
1 package functions;
2
3 public interface TabulatedFunction extends Function, Cloneable {
4     // Задание 4: Добавляем метод clone() в интерфейс
5     Object clone();
6
7     int getPointsCount();
8     FunctionPoint getPoint(int index);
9     void setPoint(int index, FunctionPoint point)
10         throws InappropriateFunctionPointException, FunctionPointIndexOutOfBoundsException;
11     double getPointX(int index) throws FunctionPointIndexOutOfBoundsException;
12     void setPointX(int index, double x)
13         throws InappropriateFunctionPointException, FunctionPointIndexOutOfBoundsException;
14     double getPointY(int index) throws FunctionPointIndexOutOfBoundsException;
15     void setPointY(int index, double y) throws FunctionPointIndexOutOfBoundsException;
16
17     void deletePoint(int index)
18         throws FunctionPointIndexOutOfBoundsException, IllegalStateException;
19     void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
20 }
```

Задание 5

=== Тестирование методов Object для табулированных функций ===

--- Задание 1: FunctionPoint ---

```
1. toString():
  p1 = (1.0; 2.0)
  p2 = (1.0; 2.0)

2. equals():
  p1.equals(p2) = true
  p1.equals(p3) = false

3. hashCode():
  p1.hashCode() = -32504895
  p2.hashCode() = -32504895
  p3.hashCode() = 525249

4. clone():
  p1.clone() = (1.0; 2.0)
  p1 == p1.clone() = false
  p1.equals(p1.clone()) = true
```

--- Задание 2: ArrayTabulatedFunction ---

```
1. Создание функций и toString():
  func1 = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}
  func2 = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}

2. equals():
  func1.equals(func2) = true
  func1.equals(func3) = false

3. hashCode():
  func1.hashCode() = -32504894
  func2.hashCode() = -32504894
  func3.hashCode() = 2114978754

4. clone() и глубокое клонирование:
  clone = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}
  После изменения func1[1].y = 100:
  func1 = {(0.0; 0.0), (1.0; 100.0), (2.0; 4.0)}
  clone = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}
  func1.equals(clone) = false
```

--- Задание 3: LinkedListTabulatedFunction ---

```
1. Создание функций и toString():
  func1 = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}
  func2 = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}

2. equals():
  func1.equals(func2) = true

3. hashCode():
  func1.hashCode() = -32504894
  func2.hashCode() = -32504894

4. clone() и глубокое клонирование:
  clone = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}
  После изменения func1[1].y = 200:
  func1 = {(0.0; 0.0), (1.0; 200.0), (2.0; 4.0)}
  clone = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}
  func1.equals(clone) = false
```

--- Задание 4-5: Сравнение и клонирование ---

```
1. Сравнение разных реализаций:
  arrayFunc = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}
  listFunc  = {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0)}
  arrayFunc.equals(listFunc) = true

2. Согласованность equals() и hashCode():
  arrayFunc.hashCode() = -32504894
  listFunc.hashCode()  = -32504894

3. Проверка изменения хэш-кода при небольшом изменении:
  arrayFunc.hashCode() (исходный) = -32504894
  arrayFunc.hashCode() (после +0.001) = 1787083824

4. Проверка метода clone() в интерфейсе:
  tabFunc через интерфейс = {(0.0; 0.0), (1.0; 1.001), (2.0; 4.0)}
  cloned через интерфейс  = {(0.0; 0.0), (1.0; 1.001), (2.0; 4.0)}
```